

ISIS-2111 PLE

Proyecto 4

VeriLang en Kotlin

Amelia Serrano Arango – 202221556
Cristian David Ortiz Sanchez – 202311404

Mayo 2026

Resumen

En este proyecto se tomó la implementación de VeriLang del Proyecto 3 (con las correcciones aplicadas de la retroalimentación de la TA) y se adaptó para ejecutarse desde un entorno Kotlin en lugar de Java. Para ello se creó el módulo `RunnerJson.rsc`, que orquesta el pipeline de análisis (parsing, construcción del AST, verificación semántica y de tipos, y evaluación de expresiones) y produce un único objeto JSON. Una aplicación de escritorio en Kotlin con Compose Desktop invoca a Rascal, extrae el JSON resultante y lo despliega en una interfaz gráfica que muestra el estado del parser, la lista de módulos, los errores encontrados y la salida del programa. Se añadió soporte para `defstruct` y `defdata` en la gramática, el AST, el parser y el checker. El `.zip` de entrega incluye tanto el código fuente de Rascal como la aplicación Kotlin.

Índice

Resumen	1
1. Introducción	3
2. Correcciones aplicadas del Proyecto 3	3
3. Gramática formal	4
3.1. Alfabeto (Σ)	4
3.1.1. Palabras reservadas	4
3.1.2. Símbolos y operadores	4
3.2. No terminales (N)	4
3.3. Símbolo inicial (S_0)	4
3.4. Reglas de producción (P)	4
4. Arquitectura	6
5. Componentes	6
5.1. Syntax.rsc	6
5.2. AST.rsc	7
5.3. Parser.rsc	7
5.4. Checker.rsc	8
5.5. Interpreter.rsc	8
5.6. RunnerJson.rsc	9
5.7. Aplicación Kotlin	9
6. Pruebas	11
6.1. test.vl (archivo válido)	11
6.2. error_test.vl (archivo con errores)	12
7. Ejecución	12
7.1. Requisitos	12
7.2. Comandos	13
7.3. Ejemplo	13
8. Conclusiones	13

1. Introducción

VeriLang es un lenguaje de programación educativo diseñado en el marco de la materia ISIS-2111 PLE para explorar conceptos de definición de lenguajes usando Rascal como *language workbench*. En los proyectos anteriores se definió la gramática formal, se implementó el parser, el AST, el intérprete y el checker semántico, y se construyó una interfaz en Java.

Para el Proyecto 4, el objetivo es migrar la interfaz a Kotlin, aprovechando Rascal para producir un JSON intermedio que la aplicación Kotlin consume. Específicamente se debe:

- Reutilizar la implementación de VeriLang del Proyecto 3 con las correcciones aplicadas.
- Extraer el AST del lenguaje a un archivo JSON mediante un módulo Rascal (`RunnerJson.rsc`).
- Crear un servicio Kotlin (`VeriLangService.kt`) que invoque a Rascal, capture la salida JSON y la deserialice.
- Construir una interfaz gráfica en Compose Desktop (`MainWindow.kt`) que muestre el estado del parser, la lista de módulos, los errores y la salida.
- Entregar un `.zip` con la solución y el documento.

Adicionalmente, se incorporaron las estructuras `defstruct` y `defdata` a la gramática y al sistema de tipos, completando así el conjunto de constructos solicitados en iteraciones anteriores.

2. Correcciones aplicadas del Proyecto 3

Atendiendo la retroalimentación de la TA (Carla) sobre la Entrega 2, se aplicaron las siguientes correcciones en el Proyecto 3, las cuales se mantienen y heredan en este Proyecto 4. El detalle completo se encuentra en el documento *Correcciones aplicadas a VeriLang.pdf*.

- **Identificadores con mayúsculas:** la regla léxica `Id` se amplió de `[a-z]` a `[a-zA-Z]` para permitir identificadores como `Set`, `Bool` o `Nat`.
- **Jerarquía aritmética completa:** se insertaron los niveles `AddExpr`, `MultExpr` y `PowExpr` entre `EqExpr` y `UnaryExpr`, añadiendo los operadores `+`, `-`, `*`, `/`, `%` y `**`.
- **Operador de equivalencia lógica:** se añadió `≡` (equivalencia) a `EqExpr`.
- **Literales en Atom:** se conectaron `IntLiteral`, `FloatLiteral` y `CharLiteral` como alternativas de `Atom`, y se añadieron `BoolLiteral` y `StringLiteral`.
- **CharLiteral con mayúsculas:** se amplió de `[a-z]` a `[a-zA-Z]`.
- **Evaluador completo:** se implementó un evaluador que reduce expresiones a valores concretos (`intVal`, `floatVal`, `boolVal`, `charVal`, `stringVal`, `appVal`), usando pattern matching sobre reglas definidas por el usuario.

3. Gramática formal

A continuación se presenta la especificación formal actualizada de VeriLang, que incorpora `defstruct` y `defdata`.

3.1. Alfabeto (Σ)

3.1.1. Palabras reservadas

```
defmodule using defspace defoperator defexpression defrule
defvar
defstruct defdata end forall exists in defer neg
or and true false Int Float Bool Char String
```

3.1.2. Símbolos y operadores

```
( ) [ ] : . , + - * / % ** < > <= >= <> = -> =>
or and neg in ≡
a..z A..Z 0..9
```

3.2. No terminales (N)

$N = \{ \text{Module, Definition, Using, SpaceDef, SpaceParent, StructDef, StructField, DataDef, DataVariant, DataVariantSignature, OperatorDef, VarDef, VarDecl, RuleDef, ExpressionDef, Type, LogicalExpression, OrExpr, AndExpr, EqExpr, AddExpr, MultExpr, PowExpr, UnaryExpr, Atom, Application, AttributeList, Attribute, AttributeValue, Letter, LowerLetter, UpperLetter, Number, Identifier, IntLiteral, FloatLiteral, CharLiteral, BoolLiteral, StringLiteral} \}$

3.3. Símbolo inicial (S_0)

$$S_0 = \text{Module}$$

3.4. Reglas de producción (P)

Para facilitar la lectura, las reglas se presentan agrupadas por categoría. Las palabras entre comillas dobles son símbolos terminales.

Estructura del módulo

```
Module ::= "defmodule" Identifier { Definition } "end"
Using   ::= "using" Identifier
```

Definiciones

```

Definition      ::= Using | SpaceDef | StructDef | DataDef
                  | OperatorDef | VarDef | RuleDef | ExpressionDef

SpaceDef        ::= "defspace" Identifier [ "<" Identifier ] "end"
StructDef       ::= "defstruct" Identifier { StructField } "end"
StructField     ::= Identifier : Type
DataDef         ::= "defdata" Identifier { DataVariant } "end"
DataVariant     ::= Identifier [ DataVariantSignature ]
DataVariantSig ::= : { Type -> }+
OperatorDef     ::= "defoperator" Identifier : { Type -> }+ [ AttributeList ] "end"
VarDef          ::= "defvar" { VarDecl ", " }+ "end"
VarDecl         ::= Identifier : Type
RuleDef         ::= "defrule" Application -> Application "end"
ExpressionDef   ::= "defexpression" LogicalExpression [ AttributeList ] "end"

```

Expresiones

```

LogicalExpression ::= OrExpr
OrExpr            ::= AndExpr { "or" AndExpr }
AndExpr           ::= EqExpr { "and" EqExpr }
EqExpr            ::= AddExpr { ("=" | "<>" | "<" | ">" | "<=" | ">=" | "≡" | "=")
AddExpr           ::= MultExpr { ("+" | "-") MultExpr }
MultExpr          ::= PowExpr { ("*" | "/" | "%") PowExpr }
PowExpr           ::= UnaryExpr { "**" UnaryExpr }
UnaryExpr         ::= [ "neg" ] Atom | "forall" Id "in" Id "." LogicalExpression
                  | "exists" Id "in" Id "." LogicalExpression
Atom              ::= Identifier | IntLiteral | FloatLiteral | CharLiteral
                  | BoolLiteral | StringLiteral | Application | "(" LogicalExpression
Application       ::= "(" Identifier { LogicalExpression } ")"

```

Elementos léxicos

```

Type              ::= "Int" | "Float" | "Bool" | "Char" | "String" | Identifier
AttributeList     ::= "[" { Attribute } "]"
Attribute         ::= Identifier [ ":" Identifier ]
Identifier        ::= Letter { Letter | Number | "-" }
Letter            ::= LowerLetter | UpperLetter
LowerLetter       ::= "a" | "b" | ... | "z"
UpperLetter       ::= "A" | "B" | ... | "Z"
Number            ::= "0" | "1" | ... | "9"
IntLiteral        ::= Number { Number }
FloatLiteral      ::= Number { Number } "." Number { Number }
CharLiteral       ::= "'" Letter "'"
BoolLiteral       ::= "true" | "false"
StringLiteral     ::= "'" { cualquier caracter no "'" } "'"

```

4. Arquitectura

La comunicación entre Rascal y Kotlin sigue el pipeline ilustrado en la Figura 1. Rascal se encarga de todo el análisis del lenguaje (parsing, AST, verificación, evaluación) y produce un único objeto JSON. La aplicación Kotlin invoca a Rascal como un proceso externo, captura el JSON por salida estándar y lo despliega en la interfaz gráfica.

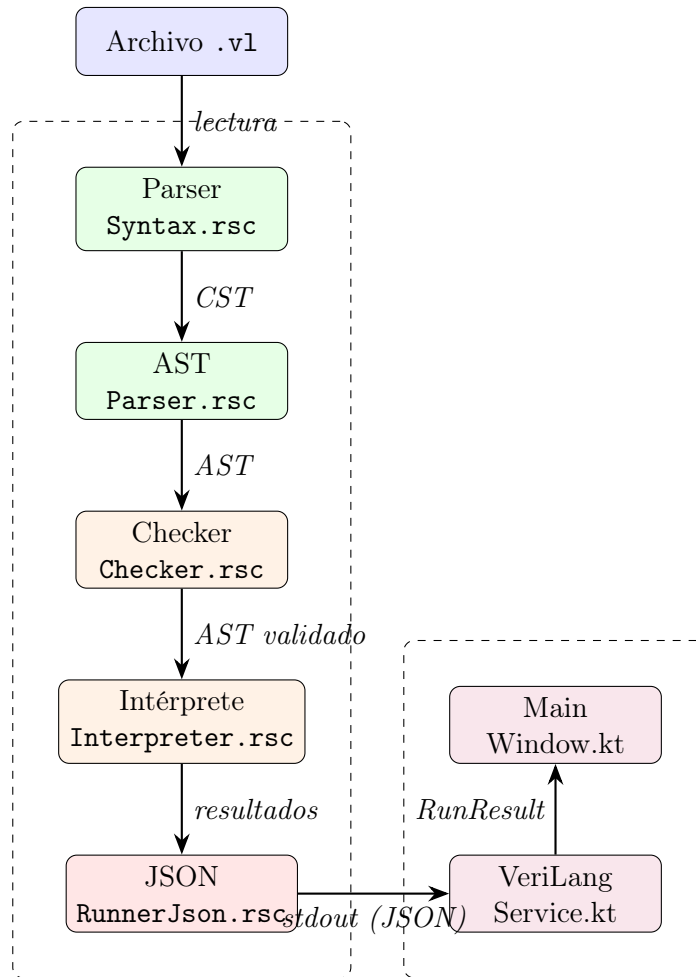


Figura 1: Pipeline de comunicación Rascal → JSON → Kotlin.

5. Componentes

5.1. Syntax.rsc

Define la gramática libre de contexto de VeriLang usando la notación de sintaxis concreta de Rascal. Incluye las reglas para `defstruct` y `defdata`:

```
syntax StructDef = structDef:
  "defstruct" Id name StructField* fields "end";
syntax StructField = structField:
```

```

    Id name ":" Type tp;

syntax DataDef = dataDef:
    "defdata" Id name DataVariant* variants "end";
syntax DataVariant = dataVariant:
    Id name DataVariantSignature? sig;
syntax DataVariantSignature = dataVariantSignature:
    ":" {Type "-\>"}+ typeSig;

```

Además se añadieron `BoolLiteral` y `StringLiteral` como alternativas léxicas, y se actualizó el conjunto `Reserved` con `defstruct`, `defdata`, `true`, `false`, `Int`, `Float`, `Bool`, `Char`, `String`.

5.2. AST.rsc

Define las estructuras de datos del árbol de sintaxis abstracta. Se añadieron los constructores para las nuevas definiciones:

```

data Definition
= ...
| structDefinition(StructDef structDecl)
| dataDefinition(DataDef dataDecl)
;

data StructDef = structNode(str name, list[StructField] fields);
data StructField = structFieldNode(str name, Type tp);
data DataDef = dataNode(str name, list[DataVariant] variants);
data DataVariant = dataVariantNode(str name, list[Type] args);

```

El tipo `Literal` unifica todos los literales:

```

data Literal
= intLiteral(int intVal) | floatLiteral(real floatVal)
| charLiteral(str charVal) | boolLiteral(bool boolVal)
| stringLiteral(str stringVal)
;

```

5.3. Parser.rsc

Transforma el árbol de parse concreto (CST) producido por Rascal en el AST definido en `AST.rsc`. Se añadieron las funciones de transformación para `StructDef`, `DataDef`, `BoolLiteral` y `StringLiteral`. El parser usa pattern matching sobre la sintaxis concreta de Rascal para cada producción.

Ejemplo para `StructDef`:

```

private AST::StructDef toStructDef(Tree tree) {
    switch (tree) {
        case (StructDef) 'defstruct <Id name> <StructField* fields> end':

```

```

    return AST::structNode(text(name),
                           [toStructField(field) | field <- fields]);
  default:
    throw "Unsupported struct definition parse tree: <tree>";
}
}

```

5.4. Checker.rsc

El checker combina dos niveles de verificación:

1. **TypePal:** mediante `extend analysis::typepal::TypePal`, se recolectan definiciones y usos de espacios, operadores y variables, y se resuelven las restricciones de tipos automáticamente.
2. **Validación manual:** se implementaron funciones específicas para detectar errores que TypePal no cubre directamente:
 - Tipos duplicados (`duplicateErrors`)
 - Tipos desconocidos en campos de `defstruct`, argumentos de `defdata`, variables y operadores (`checkTypeExists`)
 - Operadores con aridad insuficiente (`checkOperatorShape`)
 - Coherencia de tipos en reglas (`checkRule`)
 - Existencia del espacio padre (`checkSpaceParent`)

El chequeo de tipos de expresiones (`typeOf`) verifica cada constructor del AST:

- **Literales:** infieren su tipo directamente.
- **Variables:** se resuelven contra el entorno de variables.
- **Aplicaciones:** se verifica aridad y tipos de argumentos contra la firma del operador.
- **Operadores aritméticos:** requieren `Int` o `Float` con tipo coincidente; `%` solo acepta `Int`.
- **Potencia:** base y exponente deben ser `Int`.
- **Comparaciones:** `=` y `<>` requieren tipos iguales; `≡` y `=>` requieren `Bool`; `<`, `>`, `<=`, `>=` requieren `Int` o `Float`; `in` verifica compatibilidad con el dominio.

5.5. Interpreter.rsc

Evalúa las expresiones del AST a valores concretos (`RuntimeValue::Val`). Soporta:

- Literales, variables y aplicaciones de operadores.
- Reglas definidas por el usuario (`defrule`) mediante pattern matching (`RuleMatcher.rsc`).

- Operaciones aritméticas, comparaciones, potencia, negación, disyunción y conjunción.
- Cuantificadores (`forall`, `exists`) con entorno de variables acotado.

La evaluación se realiza sólo si el checker no reporta errores, lo que garantiza que el programa está bien tipado antes de ejecutarse.

5.6. RunnerJson.rsc

Es el módulo central de orquestación. Su función `main` recibe la ruta de un archivo `.vl` y ejecuta el pipeline completo:

1. Lee el archivo fuente.
2. Parsing con `parse(#start[Module], src)`.
3. Construcción del AST con `Parser::toAST`.
4. Verificación con `Checker::check` (TypePal + manual).
5. Evaluación de expresiones con `Interpreter::eval` (solo si no hay errores).
6. Impresión de un único objeto JSON con todos los resultados.

Estructura del JSON producido:

```
{
  "success": true,           // true si todo ok
  "module": "demo",         // nombre del módulo
  "modules": ["demo"],      // lista de módulos
  "parseOk": true,          // estado del parser
  "typeCheckOk": true,      // estado del type checker
  "semanticOk": true,       // estado semántico
  "typeErrors": [],         // errores de tipo
  "semanticErrors": [],     // errores semánticos
  "output": ["resultado"],   // salida de la evaluación
  "error": "",              // mensaje de error general
  "codigoFormateado": "...", // código fuente original
  "resumen": "modulo=demo; definiciones=9; expresiones=1"
}
```

5.7. Aplicación Kotlin

La aplicación de escritorio está construida con Compose Desktop y consta de cuatro archivos:

Main.kt

Punto de entrada. Crea una ventana con título `VeriLang` de 800×600 píxeles e inicia `MainWindow`.

RunResult.kt

Modelo de datos serializable con `kotlinx.serialization`. Sus campos coinciden exactamente con las claves del JSON producido por `RunnerJson.rsc`:

```
@Serializable
data class RunResult(
    val success: Boolean = false,
    val module: String = "",
    val modules: List<String> = emptyList(),
    val parseOk: Boolean = false,
    val typeCheckOk: Boolean = false,
    val semanticOk: Boolean = false,
    val typeErrors: List<String> = emptyList(),
    val semanticErrors: List<String> = emptyList(),
    val output: List<String> = emptyList(),
    val error: String = "",
    val codigoFormateado: String = "",
    val resumen: String = ""
)
```

VeriLangService.kt

Servicio que ejecuta Rascal como un proceso externo:

```
val cmd = listOf(
    "java",
    "-Dfile.encoding=UTF-8",
    "-Drascal.projectPath=${projectRoot.absolutePath}",
    "-jar", rascalJar.absolutePath,
    "RunnerJson",
    filePath
)
val process = ProcessBuilder(cmd).directory(srcDir).start()
```

Captura el stdout, extrae el primer objeto JSON válido que contenga la clave `success`, y lo deserializa en `RunResult`. Se ejecuta en `Dispatchers.IO` para no bloquear la interfaz.

MainWindow.kt

Interfaz gráfica que contiene:

- Campo de texto para la ruta del archivo y botón *Buscar* (usando `JFileChooser` filtrado a `.vl`).
- Botón *Correr* que invoca `VeriLangService.run`.
- Panel de resultados con indicadores de estado coloreados (Parse, Types, Semántica), lista de módulos, resumen del AST, errores de tipos, errores semánticos, código formateado y salida del programa.

Distribución de la interfaz

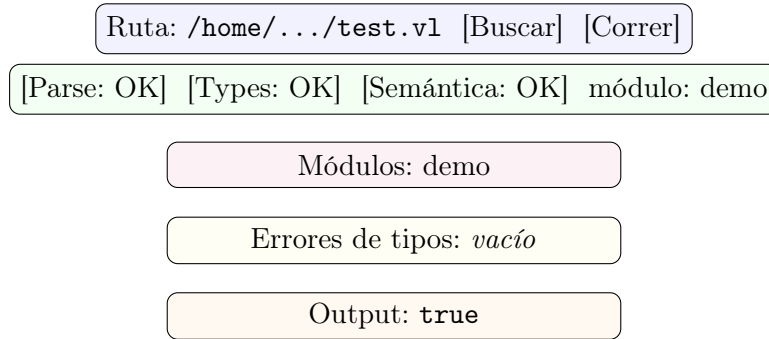


Figura 2: Esquema de la interfaz gráfica de MainWindow.kt.

6. Pruebas

Se incluyen dos archivos de prueba en el directorio `instance/`.

6.1. test.vl (archivo válido)

```
defmodule demo
using math
defspace expr end
defspace nat < expr end
defstruct pair
  first : nat
  second : nat
end
defdata truth
  yes
  no
end
defoperator plus : nat -> nat -> nat [commutative associative] end
defoperator leq : nat -> Bool [precedence:low] end
defvar x : nat, y : nat, z : nat end
defrule (plus x y) -> (plus y x) end
defexpression forall x in nat . exists y in nat . neg (x = y) end
end
```

Resultado esperado:

- `parseOk = true`
- `typeCheckOk = true`
- `semanticOk = true`

- output contiene el resultado de evaluar la expresión (un valor booleano).
- modules = ["demo"]

6.2. error_test.vl (archivo con errores)

```
defmodule errortest
defspace nat end
defstruct bad-struct
  field : missing-struct-type
  field : nat          // duplicado
end
defdata bad-data
  bad-constructor : missing-constructor-type
  bad-constructor // duplicado
end
defoperator plus : nat -> nat -> nat end
defoperator leq : nat -> nat -> Bool end
defvar x : nat end
defrule (plus x y) -> (minus x y) end // minus no existe
defexpression neg (x = z) end          // z no definida
defexpression (undefined-op x y) end    // operador no existe
defvar bad : missing-space end          // tipo no existe
defspace child < missing-parent end     // padre no existe
defexpression forall n in missing-domain . (leq n x) end
defexpression (plus x true) end         // tipo incorrecto
end
```

Resultado esperado:

- parseOk = true
- typeCheckOk = false
- semanticOk = false
- typeErrors contiene al menos 8 errores (campos duplicados, tipos desconocidos, constructores duplicados, operador no definido, variable no definida, espacio faltante, padre faltante, dominio faltante, tipo incorrecto en argumento).
- output = [] (no se evalúa si hay errores).

7. Ejecución

7.1. Requisitos

- **Rascal:** rascal-shell-stable.jar ubicado en el directorio verilang/ o en el directorio padre del proyecto.

- **Kotlin:** Gradle instalado (`gradle -version`).

7.2. Comandos

Desde la terminal, ubicarse en `verilang/kotlin-app` y ejecutar:

```
gradle run
```

La aplicación invoca internamente:

```
java -Dfile.encoding=UTF-8 \\  
      -Dascal.projectPath=<verilang> \\  
      -jar rascal-shell-stable.jar \\  
      RunnerJson <archivo.vl>
```

La primera ejecución descarga las dependencias de Compose (~2 minutos). Las siguientes son inmediatas.

7.3. Ejemplo

1. Abrir la aplicación con `gradle run`.
2. Presionar *Buscar* y seleccionar `instance/test.vl`.
3. Presionar *Correr*.
4. Los indicadores *Parse*, *Types* y *Semántica* se muestran en verde (OK).
5. El módulo `demo` aparece en la lista de módulos.
6. La salida de la evaluación se muestra en la sección *Output*.

8. Conclusiones

En este proyecto se logró migrar exitosamente la interfaz de VeriLang de Java a Kotlin, demostrando la flexibilidad de Rascal como *language workbench* para separar el análisis del lenguaje de la presentación de resultados.

Logros principales

- El pipeline Rascal → JSON → Kotlin funciona de extremo a extremo, permitiendo que cualquier lenguaje definido en Rascal se conecte a una GUI Kotlin con solo implementar `RunnerJson.rsc`.
- La gramática de VeriLang se completó con `defstruct` y `defdata`, cubriendo todos los constructores solicitados durante el semestre.
- El checker combina TypePal para resolución automática de tipos con validación manual para reglas específicas del lenguaje, resultando en un sistema de tipos robusto.

Limitaciones

- Los errores de tipo y semánticos se reportan en una sola lista, sin diferenciación. Una mejora futura sería separarlos para dar retroalimentación más precisa al usuario.
- El intérprete maneja cuantificadores de forma simplificada (asigna `appVal` a las variables cuantificadas), lo que limita la expresividad de la evaluación.
- La aplicación requiere Gradle instalado; no se incluyen los scripts `gradlew` por restricciones de tamaño del correo corporativo.

Trabajo futuro

- Separar los errores del checker en categorías (tipo vs. semánticos) para mejorar la claridad en la interfaz.
- Implementar un pretty printer que produzca código formateado a partir del AST.
- Agregar resaltado de sintaxis (`@category`) en la gramática de Rascal, que no pudo incluirse por conflictos con el sistema de palabras reservadas.
- Extender el intérprete para manejar semántica más compleja (evaluación de `defstruct`, `defdata` y constructores con argumentos).